

FACULTAD DE INGENIERÍA Y CIENCIAS

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Informe de diseño

Torneo de Komikan Concurrente

Tarea 2 – Sistemas Operativos

Implementación en C17 con hilos POSIX (pthreads)

Docente: Yerko Ortiz

Integrantes: Vicente Álvarez

Xiomara Vidalon

Fecha: 3 de noviembre de 2026

Universidad Diego Portales

Índice

1. Introducción **2**

2. Arquitectura y sincronización **2**
2.1. Modelo de ejecución 2
2.2. Acceso al tablero 2
2.3. Supervisión y cierre 2

3. Riesgos de concurrencia **2**
3.1. Bloqueos y progreso 2
3.2. Abandonos e interrupciones 3

4. Evaluación del diseño **3**
4.1. Resultados reportados 3

5. Conclusiones **3**

1. Introducción

El presente informe describe el diseño de una solución concurrente para el Torneo de Komikan de la Tarea 2 de Sistemas Operativos. El trabajo organiza la ejecución en hilos POSIX y coordina clientes, mesas y piezas mediante un Matchmaker y un Juez por partida. Para proteger el tablero se emplean mutex, mientras que condiciones y un semáforo regulan las esperas y el máximo de mesas activas. Se busca permitir el desarrollo simultáneo de partidas sin estados inconsistentes y asegurar una terminación ordenada ante victorias, abandonos o interrupciones del servidor. A continuación, se explican la arquitectura, los riesgos considerados y las decisiones de diseño.

2. Arquitectura y sincronización

2.1 Modelo de ejecución

La solución se ejecuta en un único proceso con hilos POSIX, por lo que las partidas comparten memoria sin recurrir a IPC ni a memoria compartida del sistema operativo. El hilo principal coordina el cierre; un hilo de llegadas crea los clientes y el *Matchmaker* los empareja mediante colas FIFO separadas por rol. Cada pareja admitida ocupa una de las M mesas disponibles y recibe un Juez dedicado. Este inicializa el tablero y crea 13 hilos de pieza: uno para el Puma y doce para los Perros.

2.2 Acceso al tablero

Cada mesa tiene un mutex propio que protege las casillas, las posiciones de las piezas, las capturas y el estado de la partida. La validación y aplicación de cada movimiento ocurren bajo ese mismo bloqueo. Si varias piezas intentan ocupar el mismo destino, la primera que obtiene el mutex lo reclama; las demás observan el tablero actualizado y descartan ese movimiento. Las capturas también se resuelven dentro de la sección crítica, de modo que no hace falta bloquear casillas por separado.

Las piezas esperan entre 50 y 300 ms mediante una condición de mesa. Así liberan el mutex mientras esperan y pueden despertarse cuando termina la partida. Por otra parte, un semáforo POSIX, inicializado con M, controla la admisión a las mesas. El Matchmaker espera el cupo de forma bloqueante con un timeout de 20 ms, lo que no consume CPU de manera apreciable y le permite reaccionar a una señal de cierre. El Matchmaker protege sus colas y el estado del servidor con mutex y condiciones; cada cliente y mesa sincronizan sus estados y avisos de abandono. Un mutex adicional mantiene indivisibles las líneas de la bitácora. Para prevenir ciclos de espera, los bloqueos del servidor siguen un orden fijo (Matchmaker, luego cliente, luego mesa; y tablero antes que bitácora) y la lógica del tablero no se ejecuta mientras se conserva el mutex de la mesa.

2.3 Supervisión y cierre

El Juez revisa el estado de su mesa cada 20 ms, registra la evolución del tablero y detecta victoria, abandono, interrupción o vencimiento del tiempo máximo. Al finalizar, solicita detener las piezas, espera a que concluyan, comunica el resultado y libera los recursos de la mesa. Si un jugador abandona, el Juez declara ganador al rival y realiza la misma limpieza antes de reutilizar la mesa. Si llega `SIGINT`, el manejador solo activa una bandera segura para señales. El hilo principal deja de admitir clientes, avisa a las partidas activas y espera su cierre cooperativo; los hilos hijos bloquean la señal y no se usa `pthread_cancel`, evitando interrumpirlos mientras mantienen bloqueos.

3. Riesgos de concurrencia

3.1 Bloqueos y progreso

El riesgo de interbloqueo se reduce usando un solo mutex por tablero para validar y ejecutar movimientos, además de mantener un orden definido para los demás bloqueos. Los hilos se unen sin conservar mutex que puedan necesitar para terminar. Para evitar livelock, cada pieza obtiene su propio retardo aleatorio después de cada intento, con semilla independiente; así no persiste una secuencia de reintentos sincronizados.

La espera indefinida se aborda con colas FIFO para los roles y un límite explícito de mesas. Cada partida tiene una duración máxima de 30 segundos, por defecto; al alcanzarla, se declara empate. Entre las piezas de una misma mesa, el mutex del tablero no garantiza orden FIFO; aun así, cada pieza lo conserva solo microsegundos tras un retardo aleatorio de 50–300 ms, por lo que es improbable que alguna quede sin turnos de forma sostenida (*starvation*).

Las esperas sobre variables de condición se realizan comprobando el predicado dentro de un bucle. Cada cambio de estado notifica bajo el mutex correspondiente y el Juez usa esperas con timeout, reduciendo el riesgo de despertares perdidos o espurios.

3.2 Abandonos e interrupciones

Una carrera posible ocurre si un cliente informa su abandono justo cuando el Juez termina la partida. Para evitar usar una mesa ya liberada, esta permanece asignada hasta que sus clientes finalizan y comunican su estado bajo el mutex correspondiente. Durante el cierre global, una bandera protegida rechaza nuevas llegadas; las partidas activas reciben la solicitud de finalizar y liberan sus recursos antes de que el servidor termine.

4. Evaluación del diseño

El mutex único por tablero prioriza una implementación fácil de verificar frente a bloqueos más finos por casilla. Aunque serializa los movimientos dentro de una mesa, cada sección crítica dura microsegundos frente a los retardos de 50–300 ms, por lo que la contención se considera baja (unas 75 jugadas por segundo como máximo por mesa, con 13 piezas y un retardo medio de unos 175 ms). La concurrencia principal se obtiene ejecutando mesas independientes. El semáforo expresa directamente el máximo de partidas simultáneas, mientras que las colas FIFO se encargan del emparejamiento.

El modelo de un hilo por cliente, mesa y pieza refleja de forma directa los participantes del torneo, aunque consume memoria; por eso se asignan pilas de 256 KB a los hilos cliente. La espera temporizada permite al Juez responder en hasta 20 ms y a las piezas despertar antes de completar su retardo cuando una partida termina. Entre las limitaciones, la bitácora usa un mutex global y las piezas se mueven al azar, sin una estrategia de juego; con políticas de movimiento más complejas podría ser necesario medir nuevamente la contención. En las partidas aleatorias reportadas, el Puma suele ganar.

4.1 Resultados reportados

Se ejecutaron pruebas con N = 400 jugadores y M = 8 mesas, además de casos límite con M = 1 y M = 50, y una interrupción con N = 1000. En una muestra de 20 partidas, la duración estuvo entre 1 y 4,5 s y nunca se superó el límite de mesas. Valgrind (`--leak-check=full`) con Ctrl+C a mitad de partida terminó sin errores ni bloques sin liberar, y ThreadSanitizer y AddressSanitizer/UBSan no reportaron avisos.

5. Conclusiones

El diseño propuesto combina exclusión mutua por tablero con concurrencia entre mesas, lo que simplifica la validación de movimientos sin impedir que varias partidas avancen en paralelo. Las colas FIFO, el semáforo y las variables de condición organizan la admisión y las esperas; el Juez centraliza la finalización y la liberación de recursos. En conjunto, estas decisiones buscan equilibrar seguridad, claridad de implementación y capacidad concurrente. Las pruebas reportadas respaldan el funcionamiento bajo distintos límites de jugadores y mesas, aunque el uso de movimientos aleatorios y un mutex global para la bitácora siguen siendo aspectos susceptibles de mejora.